

confusion-matrix-print

October 2, 2024

1 Confusion Matrix

```
[18]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix , classification_report

[19]: def plot_confusion_matrix(conf_matrix, class_names):
    df_cm = pd.DataFrame(conf_matrix, index=class_names, columns=class_names)
    plt.figure(figsize=(8, 6))
    sns.heatmap(df_cm, annot=True, fmt='d', cmap='Blues')
    plt.ylabel('Actual')
    plt.xlabel('Predicted')
    plt.show()
```

Here is an explanation of each line in the `plot_confusion_matrix` function:

1. Function Definition

```
def plot_confusion_matrix(conf_matrix, class_names):
```

- This defines a function named `plot_confusion_matrix` that takes two parameters:
 - `conf_matrix`: The confusion matrix, which is typically a 2D array (like a NumPy array) containing the number of predictions for each class.
 - `class_names`: A list of class names corresponding to the labels in the confusion matrix (e.g., ["Dog", "Not a dog"]).

2. Creating a DataFrame

```
df_cm = pd.DataFrame(conf_matrix, index=class_names, columns=class_names)
```

- This line converts the confusion matrix (`conf_matrix`) into a Pandas DataFrame. Using a DataFrame makes it easier to visualize the matrix with labeled rows and columns.
- `index=class_names` and `columns=class_names` set the row and column labels of the DataFrame using the list `class_names`, making the matrix easier to interpret.

3. Creating a Plot Figure

```
plt.figure(figsize=(8, 6))
```

- This line creates a new figure for the plot with a specified size (8 inches wide and 6 inches tall). This sets up the area where the heatmap will be drawn.

4. Drawing the Heatmap

```
sns.heatmap(df_cm, annot=True, fmt='d', cmap='Blues')
```

- This line uses Seaborn's `heatmap` function to create a visual representation of the confusion matrix (`df_cm`):
 - `df_cm`: The DataFrame to be visualized.
 - `annot=True`: Displays the values in each cell of the matrix.
 - `fmt='d'`: Formats the annotations as integers.
 - `cmap='Blues'`: Specifies the color scheme for the heatmap. The 'Blues' color palette is used to differentiate between higher and lower values visually.

5. Adding Axis Labels

`plt.ylabel('Actual')` `plt.xlabel('Predicted')` “– These lines set the labels for the y-axis (Actual) and x-axis (Predicted) of the plot. This indicates that the rows represent the actual classes, and the columns represent the predicted classes.

6. Displaying the Plot

```
plt.show()
```

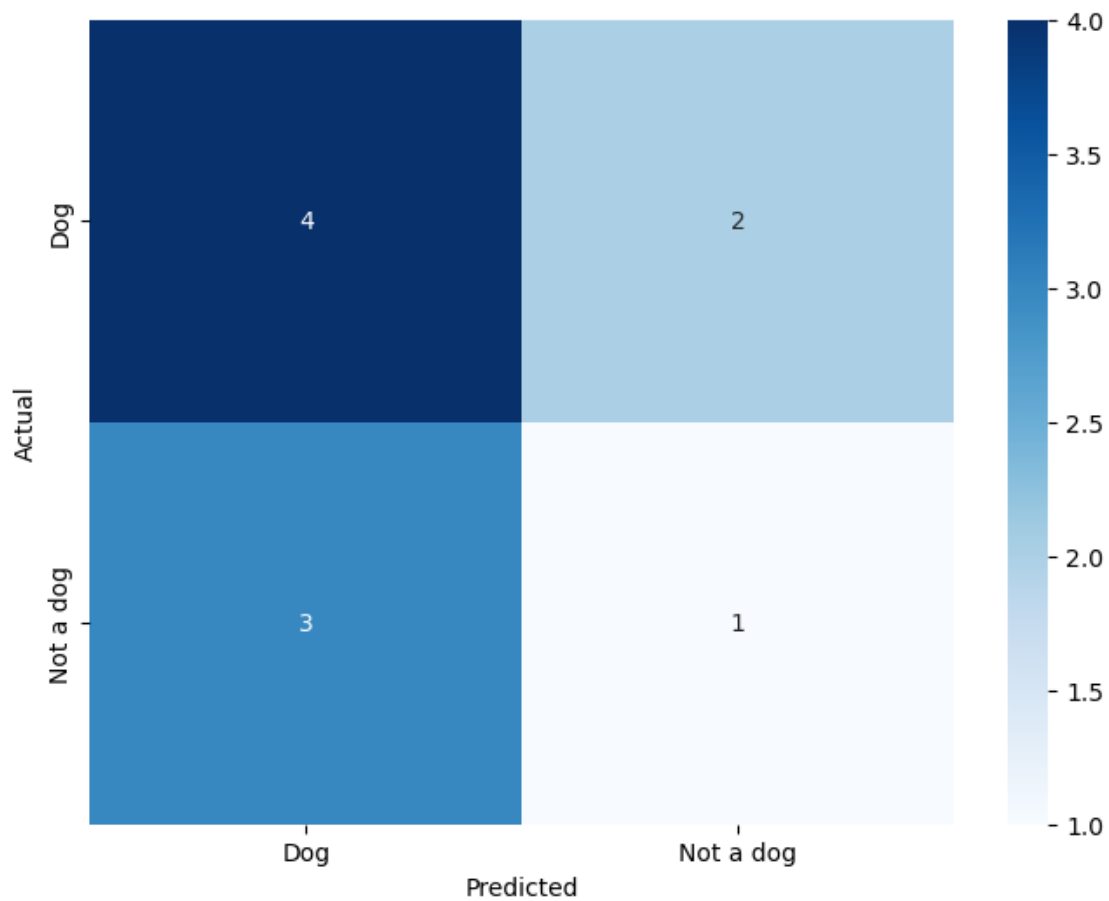
- This line displays the heatmap plot. Without this line, the plot might not render, especially in some interactive environments like Jupyter Notebooks.

Summary The `plot_confusion_matrix` function takes a confusion matrix and a list of class names, then creates a heatmap using Seaborn to visualize the matrix. It labels the rows and columns, uses a color gradient to represent values, and annotates each cell with the corresponding numbers for easy interpretation. This is helpful for understanding the performance of a classification model by showing how many instances were correctly or incorrectly classified.

```
[20]: truth = ["Dog","Not a dog","Dog","Dog", "Dog", "Not a dog", "Not a dog", "Not a dog", "Not a dog", "Not a dog"]
      ↪ "dog", "Dog", "Dog", "Not a dog"]
      prediction = ["Dog","Dog", "Dog","Not a dog","Dog", "Not a dog", "Dog", "Not a dog", "Dog", "Not a dog"]
      ↪ "Not a dog", "Dog", "Dog"]
```

```
[21]: from sklearn.metrics import confusion_matrix

      # Assuming 'truth' and 'prediction' are defined earlier in your code
      cm = confusion_matrix(truth, prediction)
      plot_confusion_matrix(cm, ["Dog", "Not a dog"])
```



```
[22]: print(classification_report(truth, prediction))
```

	precision	recall	f1-score	support
Dog	0.57	0.67	0.62	6
Not a dog	0.33	0.25	0.29	4
accuracy			0.50	10
macro avg	0.45	0.46	0.45	10
weighted avg	0.48	0.50	0.48	10

2 f1 score for “Dog” class

```
[23]: 2*(0.57*0.67/(0.57+0.67))
```

```
[23]: 0.6159677419354839
```

3 f1 score for “not a Dog” class

```
[24]: 2*(0.33*0.25/(0.33+0.25))
```

```
[24]: 0.2844827586206896
```